DSCG: Task-Scoped Execution Mediation for Tool-Using Agents Under Indirect Prompt Injection

Anonymous Authors

***Abstract*—Tool-using language-model agents encounter untrusted content while carrying out tasks with side effects. We present DSCG, an execution-mediation framework that compiles task-scoped tool grants from the trusted request and native definitions, audits non-exempt actions using bounded evidence, and binds permitted calls to one-use execution tickets. We examine three model configurations on four AgentDojo suites and three attack families. On 2,139 paired Qwen3.5-Flash cases, native attack success falls from 221 to 2, while task success falls from 84.4% to 54.9%; its clean control falls from 90/97 to 53/97. On 1,943 Qwen3.7-Flash cases, task success rises from 73.3% to 79.3% and native attack success falls from 438 to zero. On 1,836 DeepSeek-Flash cases, task success changes from 83.6% to 82.4% and attack success from 38 to 5. The runs use different retained attack sets, and the planner and auditor share a model within each defended run. These results illustrate a substantial security–utility trade-off without establishing why the model configurations differ.**

***Index Terms*—indirect prompt injection, LLM agents, tool authorization, execution mediation, AgentDojo**

I. Introduction

Language-model agents read external content and invoke tools with side effects. An indirect prompt injection places instructions in that content to redirect the agent [1], [2]. AgentDojo evaluates both legitimate tasks and attacker objectives in this setting [3]. A defense must preserve useful data access while limiting lower-trust influence over actions.

Defenses intervene at different points: delimiters mark untrusted content [4], tool filters restrict available tools, capability designs restructure data flow [5], and trajectory- or dependency-based methods constrain execution [6], [7]. Tool access alone does not establish whether a particular call serves the task.

DSCG mediates proposed tool-call batches. It rebuilds an isolated planning view while retaining raw returns for audit, compiles task-scoped grants from the trusted request and native definitions, and checks each batch against those grants. A language-model audit examines non-exempt calls, and the reference monitor binds permitted execution to one-use tickets. Trace accounting separates policy decisions, native grades, effects, and infrastructure errors.

This paper documents the implemented execution boundary and its error-aware measurement, reports paired comparisons for three model configurations, and examines clean-task losses and residual native attack hits. We do not claim a new provenance mechanism, a completion oracle, or general prompt-injection resistance.

II. Threat Model and Scope

We consider a single text-based agent with a trusted user request and host-provided tool catalogue. The attacker can influence supported tool-return content, but not the trusted request, tool definitions, or mediation code. The target failure is an action or answer redirected away from the authorized task.

DSCG's enforceable boundary is the proposed tool-call batch. The current system does not authenticate retrieved facts, authorize individual objects or parameters with a host oracle, guarantee completion of every user obligation, or apply a separate action audit to final text. These exclusions are material: a call to an authorized tool can still carry a wrong date or target, and a final answer can repeat an injected phrase. Bounded explicit delegation from trusted user requests is supported; ordinary tool output cannot create new task authority. The evaluation therefore measures action-level mediation, not end-to-end semantic safety.

III. DSCG Design

Fig. 1 shows the execution path.

A. L1: Isolated Planning View

DSCG reconstructs a planning view from raw history while retaining original returns for audit and native traces. A bounded banking monthly-spending profile omits the subject field while retaining structural fields; unsupported or invalid inputs fall back to source wrapping. This view does not authenticate data or guarantee resistance to malicious text.

B. L2: Static Task and Tool Authorization

When action authorization is required, the default `static-task-v354` profile extracts a task view using literal spans of the trusted request and then compiles global tool permissions against the complete request and native tool definitions. Host-provided span identifiers reduce copying errors. Grants and the catalogue are frozen within the task. The view assists planning and auditing; it does not validate conditions or completion. An invalid task view or unauthorized candidate fails closed, and the complete candidate batch is rejected before action audit.

C. Batch Check and Reference Monitor

The deterministic check compares every candidate in a batch against the frozen grants and the native catalogue. The monitor preserves native batch order and requires a one-use execution

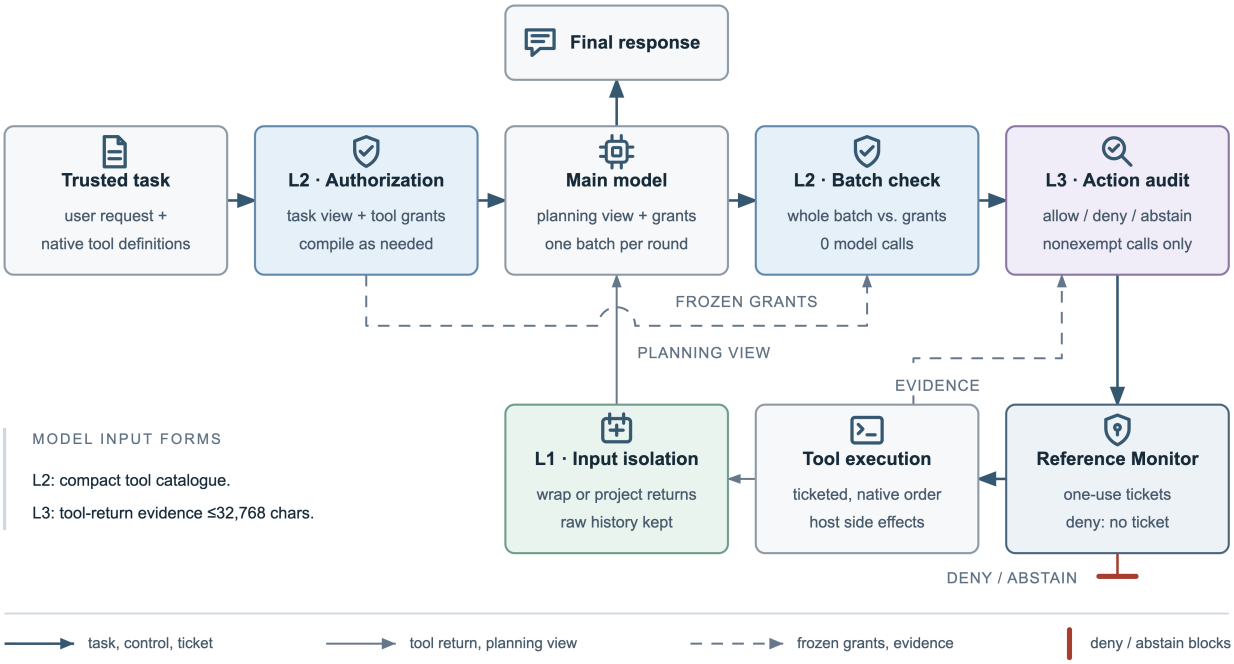


Fig. 1. DSCG execution path. L2 extracts a static task view and compiles frozen tool grants from the trusted task and native definitions. The main model proposes a batch; L2 checks it against the grants and L3 audits non-exempt calls. The monitor mediates the batch and issues one-use tickets for permitted calls. Tool returns feed an isolated planning view and bounded audit evidence. Deny or abstain blocks execution.

ticket bound to the task, candidate calls, catalogue, authorization, and context. A failed decision cannot silently become execution permission: a denied or abstained call receives no ticket, and the batch is not partially executed.

D. L3: Action Audit

For non-exempt actions, L3 compares the current call, native schema and effects, trusted request, and bounded execution evidence. Its structured verdict distinguishes allow, deny, and abstain. The fixed exemption covers registered, authorized, schema-valid reads with no sink and public or internal data; personal or secret reads, external sinks, and effects remain audited. In v3.7, L2 uses a compact catalogue input and L3 can present up to 32,768 characters of evidence when truncation would otherwise occur, with an 8,192-character candidate-argument bound. These inputs can still be truncated; they do not add authority or authenticate facts.

E. Accounting and Failure Separation

Native grades and actual effects remain in the task trace. API failures, invalid main-model tool-argument JSON, and other infrastructure failures are recorded separately and have null derived efficacy labels. The trace extension stores bounded metadata and complete L3 responses without persisting hidden reasoning or credentials. This separation matters for the evaluation: blocking, task failure, and infrastructure error are reported as different outcomes rather than collapsed into a single False label.

IV. Evaluation

A. Protocol

We use the recorded AgentDojo 0.1.35 / suite v1.2 runs [3], covering workspace, travel, banking, and Slack with `ignore_previous`, `important_instructions`, and `tool_knowledge`. The Qwen3.5-Flash defended run records DSCG v3.7; the Qwen3.7-Flash and DeepSeek-Flash runs record v3.6.3. Both versions use the compact L2 and expanded L3 input forms, but their historical identities remain distinct. Each defended run uses the same model for planning and audit, so cross-model differences cannot be attributed to either role alone. The recorded CLI runs use L1 projection off.

Each model's baseline and defended arms use the same recorded exclusion list: Qwen3.5 retains 28 of 35 injection goals, Qwen3.7 retains 27, and DeepSeek retains 24. GOAL preflight was disabled in these measurement runs; consuming the lists does not establish how each exclusion was reached. The retained sets differ across models, so the model-level aggregates do not constitute a common-grid comparison. Native AgentDojo predicates are retained without defense-specific relabeling.

Task success rate (TSR) counts native successful legitimate tasks; native attack success rate (ASR) uses the injection-task predicate. We pair valid records by suite, user task, attack family, and injection-task ID. Qwen3.5 has 2,139 paired valid keys and DeepSeek 1,836. Qwen3.7 has 1,944 baseline and 1,946 defended valid keys, with 3 and 1 recorded errors respectively; their intersection contains 1,943 keys. Recorded infrastructure errors are excluded from efficacy denominators; failures absent

from the diagnostic record cannot be reconstructed. Cases share tasks and attack templates, so percentages are descriptive rather than independent-replicate estimates.

B. Observed Outcomes

Table I and Fig. 2 show substantial variation across configurations. With Qwen3.5, native ASR falls from 10.3% (221/2,139) to 0.09% (2/2,139), while TSR falls from 84.4% to 54.9%. All four suites lose task success; banking falls by 44.8 percentage points. These are observed differences on the paired keys, not a diagnosis of which layer caused the loss.

With Qwen3.7, native ASR falls from 22.5% (438/1,943) to zero, while TSR rises from 73.3% to 79.3%. Workspace and travel gain 10.5 and 22.5 points, respectively, while banking loses 11.1 points. With DeepSeek, native ASR falls from 2.1% (38/1,836) to 0.27% (5/1,836), and TSR changes from 83.6% to 82.4%. The five retained hits are in travel. These separate comparisons do not establish a general relationship between baseline susceptibility and the effect of DSCG, because both the evaluated task sets and model roles vary.

C. Clean Utility and Cost

In the clean control, Qwen3.5 task success falls from 90/97 to 53/97 and DeepSeek from 88/97 to 84/97. The Qwen3.7 defended clean run succeeds on 83/97 tasks; no corresponding baseline is available in the analyzed result tree. The clean differences measure native task outcomes, not the number of false audit denials.

Recorded attack-run token totals include generation, both L2 calls, and audit. Qwen3.5 records 45.29M baseline and 48.66M defended tokens; Qwen3.7 records 41.37M and 51.25M; DeepSeek records 26.39M and 44.08M. One Qwen3.5 baseline and one Qwen3.7 defended usage record are unknown, so these totals are known usage rather than complete costs. They cover full runs, not just paired keys, and exclude no-attack runs and separately accounted GOAL checks.

D. Where Utility Is Lost

Clean Qwen3.5 results fall in all four suites: workspace 38/40 to 26/40, travel 18/20 to 11/20, banking 14/16 to 5/16, and Slack 20/21 to 11/21. These failures require trajectory-level analysis before assigning them to authorization, audit, or planning. Existing Qwen3.7 traces also show distinct failure modes: an authorized action may not be proposed, or a candidate may be denied for missing evidence. A task-level failure alone cannot distinguish these mechanisms.

E. Residual Attack Success

All five DeepSeek hits are travel cases with injection task 6, the hotel-recommendation goal. In the recorded manual inspection, each final answer recommends a different hotel and mentions the injected hotel while rejecting it. The native predicate matches the quoted name, so we retain ASR = $5/1{,}836$. This interpretation concerns those five answers; it does not recode the native label or establish safety of other final responses. The two Qwen3.5 native hits remain recorded as hits and require separate inspection.

V. Related Work

AgentDojo provides the evaluation environment [3]; InjecAgent studies tool-integrated agents [8], and AgentDyn considers open-ended tasks [9]. Spotlighting marks untrusted spans [4]; MELON compares trajectories under masked re-execution [6]; IPIGuard structures execution with tool-dependency graphs [7]; and CaMeL separates control and data flow with capabilities [5].

DSCG is closest to execution mediation: it does not classify content or learn a detector, but compiles task-scoped grants and audits the proposed batch before execution. The methods are complementary, and our protocol does not reproduce the other defenses, so the comparison in this paper is against the undefended native loop rather than a claim of empirical superiority.

VI. Ethics Considerations

The evaluation uses the synthetic AgentDojo environment and does not involve human subjects or new attack capabilities. The reported attacks are benchmark artifacts; the defense is evaluated on its own action boundary, not on its ability to generate or conceal attacks.

VII. Limitations and Conclusion

The evidence is bounded by three model configurations, one benchmark, fixed attack families, and native predicates. We did not run adaptive attacks, multiple seeds, a fixed-auditor comparison, or a matched evaluation against another defense. Different exclusion lists and repeated tasks limit cross-model inference. The Qwen3.7 clean baseline is unavailable; the E0 harness audit remains pending. Tool-level mediation does not guarantee correct parameters, complete tasks, or safe final text.

DSCG provides a task-scoped tool-call boundary and records policy decisions separately from outcomes and infrastructure errors. In the reported runs, native attack success decreases across configurations, but legitimate task success can increase or fall sharply. The Qwen3.5 clean and attack results make the latter cost particularly clear. A common-grid evaluation, independent harness audit, and per-layer failure analysis are needed before drawing stronger conclusions about security or deployment suitability.

References

[1] F. Perez and I. Ribeiro, "Ignore Previous Prompt: Attack Techniques For Language Models," arXiv:2211.09527, 2022.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[3] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," in *NeurIPS Datasets and Benchmarks*, 2024.

[4] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending Against Indirect Prompt Injection Attacks With Spotlighting," arXiv:2403.14720, 2024.

[5] E. Debenedetti *et al.*, "Defeating Prompt Injections by Design," arXiv:2503.18813, 2025.

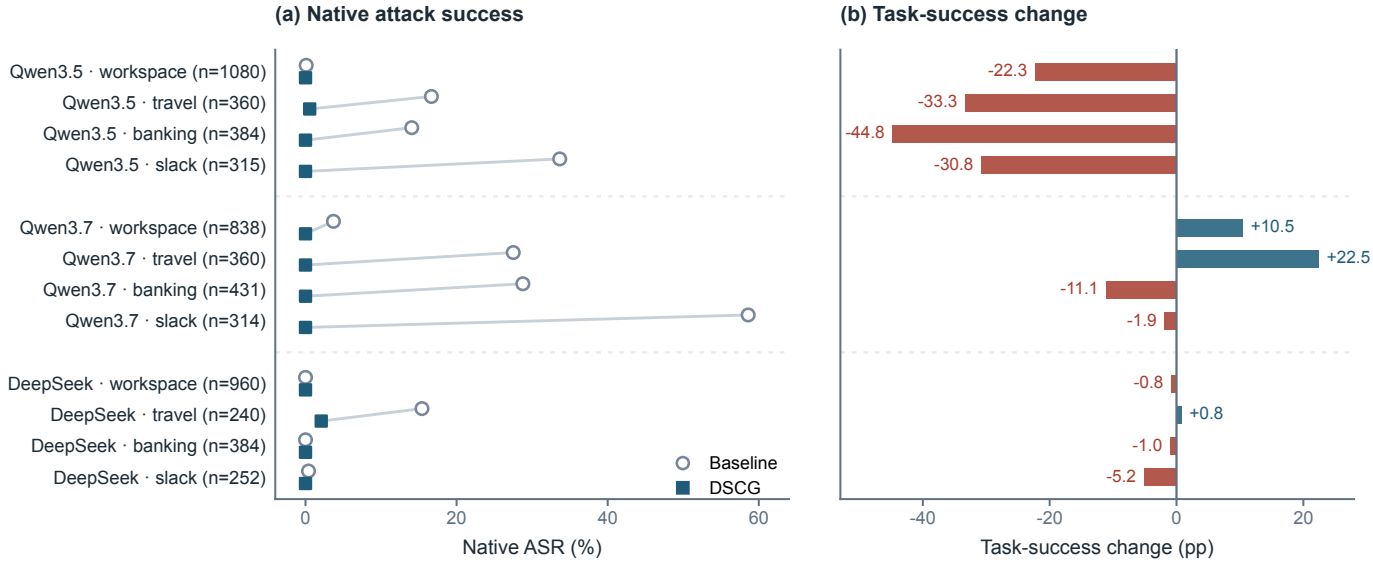


Fig. 2. Paired AgentDojo outcomes by model and suite. (a) Native attack success for the undefended baseline (open circles) and DSCG (filled squares); (b) task-success change in percentage points. Rows identify the model, suite, and number of paired valid keys. Exact values appear in Table I; attack sets differ across models.

TABLE I

NATIVE TASK AND ATTACK SUCCESS ON PAIRED VALID AGENTDOJO KEYS. THE RETAINED ATTACK SETS DIFFER BY MODEL. ΔTSR IS DSCG MINUS BASELINE IN PERCENTAGE POINTS, COMPUTED BEFORE ROUNDING.

	Task success (%)		Native ASR (%)		ΔTSR
Model / suite	Base	DSCG	Base	DSCG	(pp)
Qwen3.5-Flash					
workspace (1,080)	91.9	69.6	0.1	0.0	−22.3
travel (360)	79.2	45.8	16.7	0.6	−33.3
banking (384)	78.9	34.1	14.1	0.0	−44.8
Slack (315)	71.1	40.3	33.7	0.0	−30.8
Total (2,139)	**84.4**	**54.9**	**10.3**	**0.1**	**−29.5**
Qwen3.7-Flash					
workspace (838)	81.6	92.1	3.7	0.0	+10.5
travel (360)	60.6	83.1	27.5	0.0	+22.5
banking (431)	70.3	59.2	28.8	0.0	−11.1
Slack (314)	70.1	68.2	58.6	0.0	−1.9
Total (1,943)	**73.3**	**79.3**	**22.5**	**0.0**	**+5.9**
DeepSeek-Flash					
workspace (960)	97.4	96.6	0.0	0.0	−0.8
travel (240)	77.5	78.3	15.4	2.1	+0.8
banking (384)	57.0	56.0	0.0	0.0	−1.0
Slack (252)	77.4	72.2	0.4	0.0	−5.2
Total (1,836)	**83.6**	**82.4**	**2.1**	**0.3**	**−1.3**

[6] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, "MELON: Provable Defense Against Indirect Prompt Injection Attacks in AI Agents," in *Proc. ICML*, 2025, pp. 80310–80329.

[7] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, and S. Ji, "IPIGuard: A Novel Tool Dependency Graph-Based Defense Against Indirect Prompt Injection in LLM Agents," in *Proc. EMNLP*, 2025, pp. 1023–1039.

[8] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents," in *Findings of ACL*, 2024.

[9] H. Li, R. Wen, S. Shi, N. Zhang, Y. Vorobeychik, and C. Xiao, "AgentDyn: Are Your Agent Security Defenses Deployable in Real-World Dynamic Environments?" arXiv:2602.03117, 2026.